Project Plan: Solar Panel and Battery System

EECE 2140 Computing Fundamentals for Engineers, Iteration 1

Advaith Nair, Jailen Patel, [Teammate 3]

October 13, 2026

***Note: Teammate 3 left group during iteration 1 process; Their work will be divided evenly between the other 2 members.**

Project Scope and Expected Deliverables

Our project is a C++ console program that simulates a home solar panel and battery system hour by hour. The program reads hourly sunlight and household electricity demand from CSV files, calculates how much energy the panels produce, and decides where that energy goes. Each hour it follows the same order: solar powers the home first, extra solar charges the battery, and anything left is sent to the grid. When solar is not enough, the battery covers the gap first and the grid covers the rest. The program then reports how much of the home's demand was met by solar and the battery, and how much came from the grid.

The project does not include real hardware, live weather data, or a graphical interface.

Expected deliverables:

- C++ source code in the GitHub repository with a README explaining how to build and run it.
- Sample input files for at least a sunny day, a cloudy day, and a 7-day period.
- A test program that checks each class.
- Output files showing hourly results and a printed summary for each run.
- Iteration documents, a final report, and a final presentation.

1. Define Your Objectives

Individual Objectives

Advaith Nair

- Write the SolarPanel class, which turns hourly sunlight values into energy output using panel size and efficiency.
- Write the code that reads and checks the CSV input files for missing or invalid values.
- Set up the GitHub repository and combine everyone's code into one working build at the end of each iteration.
- Contribution: the rest of the simulation depends on solar input being correct, and integration keeps the team working from one codebase.

Jailen Patel

- Write the Battery class, which tracks how charged the battery is and keeps it within its capacity, minimum charge, and charging speed limits.
- Write and run the test program, with at least three tests for each class.
- Contribution: the battery is the main part of the balancing logic, and testing is how we show the program works.

[Teammate 3]

- Write the LoadProfile class for household demand and the Report class for output files and the summary.
- Find or create the sample input data.
- Lead documentation, including the README, iteration documents, and meeting notes.
- Contribution: makes the results readable for the team and instructor and keeps project records up to date.

All three of us will write the EnergyManager class together, since it contains the main decision logic and everyone should understand it.

Shared Team Objectives

We agreed on these objectives as a team. Each one can be checked with a test or a clear yes or no.

- The program runs a full 24-hour and 7-day simulation on every sample input file without crashing.
- Energy is balanced every hour: energy coming in equals energy going out, within 0.001 kWh.
- The battery charge never goes above its maximum or below its minimum in any test run.
- Every class has at least three passing tests before the final submission.
- The code compiles with no warnings.
- Every change is reviewed by at least one other teammate before it is merged.
- Each iteration is submitted on time.

2. Select Technologies and Tools

- **C++:** required for the course, and classes fit the project well because each class can represent one part of the system (panel, battery, demand).
- **g++ compiler:** free and available on all our computers. Compiling with warnings turned on helps us catch mistakes early.
- **VS Code:** all three of us already use it, and it has debugging and Git built in.
- **Git and GitHub:** required for submission. Separate branches let us work at the same time without overwriting each other's code.
- **GitHub Issues:** used to list tasks with an owner and due date so everyone can see what is done and what is left.
- **CSV files:** simple to read and write in C++, and they open directly in Excel or Google Sheets.

- **Excel or Google Sheets:** used to make charts from the output files without adding graphics code to the program.
- **iMessage:** used for quick questions and updates between meetings.

3. Create a Project Timeline

The dates below are rough targets. We left extra time in each phase and a full buffer period near the end so we have room to fix mistakes, rework parts that do not fit together, and handle busy weeks with other classes. We will adjust dates to match the course schedule.

Phase 1: Planning (now to about October 20)

- Tasks: write this plan, set up the repository, and agree on class names and functions. (All)
- Deliverable: project plan PDF in the repository and header files for each class.

Phase 2: Individual Classes (late October to early November, about 2 to 3 weeks)

- SolarPanel and input reading.
- Battery class and first tests.
- LoadProfile class and sample input files.
- Deliverable: each class compiles and passes its own tests.
- Milestone: all individual classes working.

Phase 3: Putting It Together (early to mid November, about 2 weeks)

- EnergyManager decision logic.
- Report class and output files.
- Combine all parts into one program.
- Energy balance and battery limit tests.
- Deliverable: a full 24-hour run from input file to output file.
- Milestone: first complete simulation.

Buffer and Rework (late November, about 1 week)

- Time set aside for fixing bugs, redoing parts that did not work as planned, and catching up on anything that slipped. This period also covers the Thanksgiving break.

Phase 4: Finishing (late November to early December)

- 7-day simulation and comparison of two battery sizes.
- Final testing with unusual and invalid inputs.
- Final README, report, and charts.

- Final presentation and demo. **(December 8, Tuesday, 10:18 A.M.)**
- Deliverable: final code, documents, and presentation.
- Milestone: final submission.

Communication, Progress Tracking, and Challenges

- We will meet once a week for about 30 to 45 minutes and check in briefly after class. The person running the meeting rotates each week.
- We will use the group chat for quick questions and try to reply within a day.
- Each task will be a GitHub Issue with an owner. Each person works on their own branch and asks a teammate to review before merging.
- If someone expects to miss a deadline, they will tell the team as early as possible so we can adjust the plan or move tasks.
- If someone is stuck for more than a day, we will work on the problem together or go to office hours.
- If we disagree on a decision, we will talk it through at the next meeting and vote if needed. Questions about requirements go to the instructor.

4. Document Your Plan

The project scope and deliverables are at the top of this document, objectives are in Section 1, tools are in Section 2, and the timeline is in Section 3. The contribution table and meeting summary are below.

Contribution Table

Team Member	Completed This Iteration	Responsibilities Next Iteration
Advaith Nair	Proposed the hourly simulation idea and energy routing order Created the GitHub repository Wrote the SolarPanel header file Drafted the scope and tools sections	Write the SolarPanel class Write the input file reader Review teammates' code
Jailen Patel	Looked up typical home battery values Wrote the Battery header file Proposed the team objectives and testing approach Drafted the objectives section	Write the Battery class Start the test program Review teammates' code

[Teammate 3]	Found example solar and demand data Wrote the LoadProfile and Report header files Set up GitHub Issues Drafted the timeline and meeting notes and put together the PDF	Write the LoadProfile class Create sample input files Write the next iteration document

5. Teamwork and Individual Contributions

We set up our plan so each part of the teamwork rubric is supported:

- **Take responsibility:** every task has a named owner, and each person owns at least one class and one team role.
- **Collaborate respectfully:** we make decisions together at weekly meetings, review each other's code, and write the main logic as a group.
- **Establish goals:** each member wrote their own objectives, and we agreed on shared objectives that can be tested.
- **Plan tasks:** tasks are split evenly across phases, with time built in for delays and setbacks.
- **Meet objectives:** progress is visible in GitHub Issues and commits, and members tell the team early about any problems.